Laboratório Prolog · Aula 13

João Marcos Moreira Nunes · RA 24055168-2 · Turma Esoft-6S-N-C
Gerado em 19/10/2026, 19:17:34 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) responde L = scheme. paradigma(java, logico) responde false, pois a base não contém esse fato nem uma regra que o prove.

Saída do ola.pl

```
Saída esperada (não executada diretamente):
?- linguagem(_).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog, _).
   paradigma(prolog, logico)
?- paradigma(_, funcional).
   paradigma(scheme, funcional)
?- paradigma(java, logico).
   false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual valor de P está associado ao Prolog e responde P = logico. paradigma(prolog, logico) pergunta se o fato específico é verdadeiro; nesse programa, a resposta é true.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

O resultado significa que o programa não consegue provar essa relação usando os fatos e regras cadastrados. Não é uma afirmação universal sobre Java; a base só registra Java como orientado_a_objetos.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas: lucia, marta, tereza, joana, bia, ana e dora. A consulta (mae(M,F), mulher(F)) terá 4 pares: (lucia, tereza), (marta, joana), (tereza, bia) e (joana, ana).

Saída do familia.pl

```
Saída esperada (não executada diretamente):
mae(joana,jake) → verdadeiro
mae(jake,joana) → false
pai(maria,jake) → false
mae(joana, _) → jake; ana
pai(_, jake) → jorge
mulher(_) → lucia; marta; tereza; joana; bia; ana; dora
(mae(joana,X), homem(X)) → X=jake
(pai(vern,Y), mulher(Y)) → Y=joana
(mae(_,Filha), mulher(Filha)) → Filha=tereza; joana; bia; ana (a ordem acompanha os fatos de mãe e mulher).
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta procura pares em que Mae seja mãe de Filha e Filha também esteja cadastrada como mulher. Há 4 respostas: (marta, joana), (tereza, bia), (joana, ana) e (joana, jake não entra, pois jake não é mulher).

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: se a base não permite provar uma afirmação, o Prolog a trata como falsa. Em situações reais, a base pode estar incompleta; uma informação ausente não significa necessariamente que seja falsa.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

A ordem é lucia, marta, tereza, joana, bia, ana e dora. O Prolog segue a ordem em que os fatos aparecem na base e faz a busca de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X,jake) responde X = paulo: paulo é irmão de joana, mãe de jake, e é homem. primo(leo,X) responde X = jake e X = ana, pois o pai de leo, paulo, é irmão de joana, mãe de jake e ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

f(X,X)=f(a,b) falha porque X teria de ser a e b ao mesmo tempo. f(X,X)=f(a,Y) unifica com X=a e Y=a. [A|B]=[] falha porque uma lista vazia não tem cabeça e cauda. 2+3=5 falha: = compara/unifica termos, não calcula a expressão.

Saídas de regras.pl e unificacao.pl

```
Saída esperada resumida (não executada diretamente):
regras.pl:
avo(X,jake) → X=vern; X=marta
avo(osvaldo,X) → X=joana; X=paulo; X=bia
irmao(jake,X) → X=ana
 tio(X,jake) → X=paulo
primo(leo,X) → X=jake; X=ana
ancestral(osvaldo,X) → vern, tereza e descendentes demonstráveis: joana, paulo, jake, ana, leo, bia
ancestral(X,jake) → jorge, joana, vern, marta, osvaldo, lucia (ordem de busca pode variar conforme o
interpretador)
ancestral(jake,X) → false
unificacao.pl: a=a unifica; a=b não unifica; variável livre com a unifica; f(X,X)=f(a,b) não unifica;
f(X,X)=f(a,Y) unifica com X=a,Y=a; pai(X,jake)=pai(jorge,Y) unifica X=jorge,Y=jake; [H|T]=[1,2,3] unifica;
[H|T]=[] não unifica; listas compatíveis unificam; 2+3=5 não unifica.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

São necessárias duas cláusulas para cobrir os dois casos: o caso base ancestral(X,Y) :- progenitor(X,Y), quando X é pai ou mãe diretamente de Y; e o caso recursivo ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y), quando X é progenitor de alguém que é ancestral de Y.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a consulta irmao(jake,X) também pode responder X = jake, porque jake compartilha os mesmos pais consigo mesmo. A meta X \== Y exclui esse caso e garante que a pessoa seja diferente do próprio indivíduo.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira expressão, a mesma variável X teria de ser igual a a e a b ao mesmo tempo, então falha. Na segunda, X = a e Y também pode ser a; assim, os dois termos ficam iguais e unificam.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, uma relação pode ser consultada em diferentes direções, desde que os argumentos e regras permitam encontrar soluções. Funções comuns de Java ou Scheme normalmente recebem entradas determinadas e calculam um resultado; não resolvem

automaticamente a relação inversa sem código adicional.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

A consulta concatena(X,Y,[1,2,3]) terá 4 respostas, uma para cada divisão da lista: X=[] e Y=[1,2,3]; X=[1] e Y=[2,3]; X=[1,2] e Y=[3]; X=[1,2,3] e Y=[].

Saída do listas.pl

```
Saída esperada (não executada diretamente):
membro(b,[a,b,c]) → true
membro(z,[a,b,c]) → false
membro(_, [a,b,c]) → a; b; c
tamanho([a,b,c],N) → N=3
tamanho([],N) → N=0
concatena([1,2],[3,4],R) → R=[1,2,3,4]
concatena(X,[3],[1,2,3]) → X=[1,2]
concatena([1],Y,[1,2,3]) → Y=[2,3]
concatena(X,Y,[1,2,3]) → ([],[1,2,3]); ([1],[2,3]); ([1,2],[3]); ([1,2,3],[])
inverte([1,2,3],R) → R=[3,2,1]
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([],0), que define o tamanho da lista vazia como zero. O passo recursivo é tamanho([_|Cauda],N) :- tamanho(Cauda,M), N is M+1. Cada chamada deixa pendente somar 1 ao tamanho calculado para a cauda.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há quatro respostas porque a lista [1,2,3] pode ser dividida em prefixo e sufixo em quatro posições, incluindo antes do primeiro elemento e depois do último. A função anexar do Scheme costuma construir uma lista a partir de entradas dadas; não procura automaticamente todos os pares de listas que concatenam no resultado.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking. A regra recursiva procura o elemento na cabeça ou continua pela cauda, permitindo ao Prolog voltar e gerar cada solução possível.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 unifica X com a expressão simbólica 2+3, sem calcular. X is 2 + 3 calcula e resulta em X=5. fatorial(N,120) provoca erro porque o cálculo usa comparações e is com N ainda não instanciado; a regra não foi escrita para descobrir N a partir do resultado.

Saída do aritmetica.pl

```
Saída esperada (não executada diretamente):
_ = 2+3 → variável unificada com o termo 2+3
_ is 2+3 → 5
5 =:= 2+3 → true
2+3 = 5 → false
_ is 7 mod 3 → 1
_ is 7 // 2 → 3
_ is 2 ^ 10 → 1024
fatorial(5,F) → F=120
fatorial(0,F) → F=1
fib(10,F) → F=55
soma([1,2,3,4],S) → S=10
fatorial(_,120) → erro de instanciação/variável não suficientemente instanciada
_ is _+1 → erro de instanciação
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= tenta unificar termos sem calcular: X = 2+3 liga X à expressão 2+3. is avalia a expressão do lado direito: X is 2+3 resulta em X=5. =:= compara valores numéricos após calcular ambos os lados: 5 =:= 2+3 é verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N,120) dá erro porque a regra usa N > 0 e N1 is N-1, operações que exigem N conhecido. concatena(X,[3],[1,2,3]) funciona porque unificação de listas pode descobrir X sem fazer cálculo aritmético; a aritmética de is não é totalmente relacional.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam caso base e recursão para calcular fatorial. Em Prolog, a relação fatorial(N,F) conecta entrada e resultado, mas as operações aritméticas exigem valores conhecidos em momentos específicos. Em Haskell, fact é uma função que recebe n e retorna o resultado, com avaliação funcional.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2 dá false porque tenta provar \+ casado(X) com X ainda livre. maximo2(5,3,M) dá duas respostas (M=5 e M=3). maximo3(5,3,3) dá true pela segunda cláusula, pois a primeira não unifica o terceiro argumento com o valor 5.

Saída de controle.pl e mensagem de ancestral_ruim

```
Saída esperada resumida (não executada diretamente):
mesmo_gosto(X,Y) encontra pares distintos que gostam da mesma bebida: ana e caio, e caio e ana.
solteiro1(X) → osvaldo, paulo, jake
solteiro2(X) → false
maximo1(5,3,M) → M=5
maximo2(5,3,M) → M=5 e M=3
maximo3(5,3,M) → M=5
maximo1(5,3,3) → false
maximo3(5,3,3) → true pela segunda cláusula
ancestral_bom(osvaldo,jake) → sim
ancestral_ruim(osvaldo,jake) → estouro da pilha (recursão à esquerda)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações válidas. Com 2 cores não existe nenhuma coloração válida para todas as fronteiras indicadas.

Saída do mapa.pl

```
Saída esperada (não executada diretamente), ordem [RS, SC, PR, SP, MS, MG, RJ, ES]:
Primeira: [vermelho, verde, vermelho, verde, azul, vermelho, azul, verde]
Segunda: [vermelho, verde, vermelho, azul, verde, vermelho, verde, azul]
Total com 3 cores: 24
Total com 2 cores: 0
Total com 4 cores: 1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog escolhe primeiro amigo para X e Y, depois exige X \== Y e procura uma bebida B de que ambos gostem. Quando uma meta falha, volta à escolha pendente mais recente e tenta o próximo amigo. Só pares distintos com gosto em comum passam: ana/caio e caio/ana.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1 primeiro gera um homem e depois verifica que ele não está em casado, por isso encontra osvaldo, paulo e jake. solteiro2 tenta negar casado(X) com X livre; como há pelo menos uma pessoa casada, a negação falha antes de gerar os homens. Por isso solteiro1 afirma que osvaldo é solteiro: ele é homem e não existe fato casado(osvaldo).

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte não protegeu esse uso porque a primeira cláusula maximo3(X,Y,X) precisa unificar também o terceiro argumento antes de executar o corpo. Na consulta maximo3(5,3,3), essa unificação falha, então o corte nunca é alcançado e a segunda cláusula dá true. Uma correção é separar o teste do resultado: maximo3(X,Y,M) :- X >= Y, !, M = X. maximo3(_,Y,Y).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim começa chamando ancestral_ruim(X,Z) antes de reduzir o problema com progenitor, repetindo a recursão sem progresso e estourando a pilha. ancestral_bom primeiro encontra um progenitor e depois chama a recursão para um problema menor, chegando ao caso base.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Há 24 colorações com 3 cores, nenhuma com 2 e 1728 com 4. O programa é declarativo porque descreve as possibilidades de cor e as restrições entre regiões; o mecanismo de backtracking do Prolog faz a busca pelas combinações que satisfazem as regras.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

concatena é uma relação: pode montar a lista resultante ou, consultada ao contrário, descobrir os prefixos e sufixos possíveis. anexar no Scheme normalmente é uma função que recebe as listas de entrada e devolve uma lista; não gera automaticamente as entradas a partir do resultado.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Uma deficiência é a aritmética não ser totalmente relacional: fatorial(N,120) falha/gera erro com N desconhecido, pois is e comparações exigem valores instanciados. Outra é a ordem da busca importar: ancestral_ruim entra em recursão infinita e estoura a pilha por chamar a si mesmo antes de reduzir o problema.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras e restrições, como relações familiares, dedução lógica e coloração de mapas, em que é útil gerar soluções possíveis. Não escolheria Prolog para uma aplicação comum com interface gráfica, integração extensa e fluxo procedural,

para a qual Java oferece ecossistema e controle mais adequados.